הרצאה 2: אלגוריתמים רקורסיביים

אלגוריתם רקורסיבי, הוא אלגוריתם המבצע קריאה לעצמו.

להלן כמה הגדרות בנושא זה:

הגדרה 2.1

יהי Alg אלגוריתם רקורסיבי, ונניח ש-Alg נקרא על ידי שיטה אחרת כלשהי.

1. הקריאה הראשונה של Alg נקראת **הקריאה הראשית.**
2. כל קריאה רקורסיבית ש-Alg מבצע נקראת **קריאת משנה**.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

3. תהי R קריאה כלשהי של Alg.

הערך ש-R מחזירה, נקרא **הערך המוחזר** של R.

4. נניח ש-Alg מבוצע עם קלט i . אם במהלך ביצוע Alg עם i , לא מבוצעות קריאות משנה, i הוא **קלט קצה** של Alg .

<u>תזכורת</u>

בהרצאה 1 הצגנו את האלגוריתם האיטרטיבי Sum הפותר את הבעיה הזו.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

Rsum **- דוגמה לאלגוריתם רקורסיבי**

כעת נציג את *Rsum* – אלגוריתם רקורסיבי הפותר את הבעיה החישובית **סכום מערך מספרים**.

```
Rsum(A)
    if (|A|=1) return(A[1])
    mid ← ⌊n/2⌋
    s_l ← Rsum(A[1,...,mid])
    s_r ← Rsum(A[mid+1,...,n])
    return(s_l + s_r)
end(Rsum)
```

איור 2.2: אלגוריתם *Rsum*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תיאור אלגוריתם *Rsum*

אלגוריתם *Rsum* מקבל כקלט מערך מספרים A. קלטי הקצה הם כל המערכים עם איבר יחיד. בתחילת הביצוע, האלגוריתם בודק האם ב-A יש איבר יחיד . אם התשובה חיובית, האיבר היחיד במערך הקלט מוחזר כפלט. אם לא, האלגוריתם מחשב את האינדקס ה"אמצעי בקרוב" של מערך הקלט, *mid*.

לאחר חישוב *mid* מתבצעות שתי קריאות רקורסיביות, עם $A[1,\ldots,mid]$ ועם $A[mid+1,\ldots,n]$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

לאחר החזרת פלטי המשנה,
האלגוריתם מחבר את הערכים
המוחזרים והסכום המתקבל הוא
הערך המוחזר מן הקריאה הראשית.
אפשר לראות שאם קריאות המשנה
מחזירות סכומים נכונים אזי
האלגוריתם מחזיר סכום נכון.
כעת נראה איך "לתרגם" הבנה זו
להוכחת נכונות פורמלית.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיום אלגוריתם *Rsum*

כדי להוכיח סיום אלגוריתם רקורסיבי עלינו להראות שכל שרשרת קריאות רקורסיבית מסתיימת בקריאה לקלט קצה.

הוכחות הסיום הן בדרך כלל פשוטות.

במקרים כאלה נסתפק נביא נימוק בלתי פורמלי לכך שלא קיים קלט שבריצה עמו מתבצעת שורת קריאות אינסופית.

לדוגמה נתבונן בהוכחת הסיום של אלגוריתם *Rsum*.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 2.3: סיום אלגוריתם *Rsum*

לכל מערך מספרים (תיאור הקלט) A, אלגוריתם *Rsum* (שם האלגוריתם) המופעל עם A מסתיים.

הוכחה

נתבונן בביצוע *Rsum* עם A. בכל קריאת משנה מספר איברי הקלט קטן. מכאן נובע שלאחר מספר קריאות רקורסיביות סופי מתבצעת קריאה עם מערך קלט ובו איבר יחיד. כל קלט ובו איבר יחיד הוא קלט קצה אשר מאינו מבצע קריאה רקורסיבית נוספת אלא מחזיר ערך מייד. מכאן: אלגוריתם *Rsum* מסתיים תמיד. **מש"ל**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 2.4 נכונות אלגוריתם *Rsum*

לכל מערך מספרים (תיאור הקלט) A,
אלגוריתם *Rsum* (שם האלגוריתם)
המופעל עם A מחזיר את סכום איברי
A (תיאור הפלט).

דיון

באלגוריתמים רקורסיביים אין
לולאות ולכן לא נוכל להשתמש
באינדוקציה על מספר המעברים
בלולאה.
במקום זאת נוכיח את המשפט
באינדוקציה **שלמה** על מספר האיברים
ב-A.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תזכורת: באינדוקציה שלמה ההנחה מתייחסת לכל מערכי המספרים שגודלם קטן מ-n ולא רק למערכים שיש להם $n-1$ איברים.

הוכחה

נוכיח את המשפט באינדוקציה (שלמה) על מספר איברי מערך הקלט A.

בסיס: ($|A|=1$)

זהו מקרה הקצה, מן הקוד נובע שאם במערך הקלט יש איבר יחיד, האלגוריתם מחזיר את האיבר היחיד ב-A, $A[1]$. ברור שהפלט נכון.

צעד האינדוקציה

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הנחה

נניח שכאשר אלגוריתם $Rsum$ מקבל כארגומנט מערך קלט שמספר איבריו קטן מ-n (כאן האינדוקציה השלמה באה לידי ביטוי) הוא מחזיר את סכום איברי מערך הקלט.

הוכחה

יהי A מערך מספרים שרירותי ובו n איברים. במהלך ביצוע $Rsum(A)$ מתבצעות קריאות המשנה

$Rsum(A[1,\ldots,mid])$

ו-$Rsum(A[mid+1,\ldots,n])$. מספר איברי $A[1,\ldots,mid]$ קטן ממש מ-n

© כל הזכויות שמורות לפרופסור עמוס ישראלי

ולכן, לפי הנחת האינדוקציה, הקריאה הרקורסיבית

$Rsum(A[mid+1,\ldots,n])$ מחזירה

את סכום איברי

$Rsum(A[mid+1,\ldots,n])$.

מאותה סיבה הקריאה הרקורסיבית

$Rsum(A[mid+1,\ldots,n])$ מחזירה

את סכום איברי $A[mid+1,\ldots,n]$.

נכונות האלגוריתם נובעת. **מש"ל**

שימו לב לפשטות הוכחת הנכונות.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מתכון להוכחת נכונות אלגוריתמים רקורסיביים

ההוכחה משתמשת **באינדוקציה על אורך הקלט**.

- **בסיס האינדוקציה**: יש להוכיח ישירות, באמצעות מעקב, כי לכל אחד מקלטי הקצה, הפלט המוחזר הוא נכון.
- **הנחת האינדוקציה:** יש להניח כי בכל קריאה לאלגוריתם עם מערך קלט שמספר איבריו קטן מ-n מוחזר פלט נכון.
- **ההוכחה:** יש להוכיח על ידי מעקב על ביצוע קריאה עם מערך קלט שבו n איברים מוחזר פלט המתאים לקלט המקורי.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

טעות נפוצה

כותבים הוכחה המבצעת מעקב מן הקריאה הראשית, ועד לקלטי הקצה. חשוב לציין כי "הוכחה" כזאת אינה יכולה להיות נכונה.

סיבוכיות אלגוריתם *Rsum*

ננתח את סיבוכיות אלגוריתם *Rsum* בשתי דרכים:

1. שיטת האיטרציה שלמדתם בקורס מבני נתונים.
2. שיטת עץ הרקורסיה – שיטה חדשה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

ניתוח הסיבוכיות בשיטת האיטרציה

נסמן ב-$t(n)$ את סיבוכיות אלגוריתם *Rsum* עבור קלט בן n איברים.

$$t(n) = c + 2t(n/2)$$

$$= c + 2c + 4t(n/4)$$

$$= c + 2c + 4c + 8t(n/8)$$

$$= \cdots$$

$$= c\left(1 + 2 + \cdots + 2^{i-1}\right) + 2^i t\left(n/2^i\right)$$

$$= c\left(1 + 2 + \cdots + 2^{\log n - 1}\right) + nt(1)$$

$$= c\left(2^{\log n} - 1\right) + nt(1) = \Theta(n)$$

ידוע ש-$t(1) = const.$

כלומר הסיבוכיות היא **לינארית.**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מושגים שנשתמש בהם בתיאור שיטת עץ הרקורסיה

הגדרה 2.5

יהי T עץ בינארי:

1. צומת l ב-T הוא **עלה** אם ל-T אין בנים.
2. צומת v ב-T הוא **צומת פנימי** אם יש לו לפחות בן אחד.
3. העץ T הוא **מלא** אם לכל צומת פנימי יש **שני בנים**.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

עץ הרקורסיה של אלגוריתם רקורסיבי

יהי Alg אלגוריתם רקורסיבי. כל ביצוע של Alg עם קלט נתון, i, משרה (induces) **עץ רקורסיה** המתאר את ביצוע Alg עם i. כל צומת בעץ הרקורסיה מייצג קריאה ל- Alg:

שורש העץ מייצג את הקריאה הראשית. כל קריאת רקורסיבית ל-Alg מתוארת על ידי קשת המובילה מן הצומת המייצג את קריאת האב, אל צומת נוסף המייצג את קריאת הבן.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

כל עלה (צומת שאין לו בנים) ב-t מתאר קריאה ל-Alg עם קלט קצה.

עץ הרקורסיה הוא כלי עזר חיוני להבנת האלגוריתם, הוכחת הנכונות, ובייחוד לחישוב הסיבוכיות שלו.

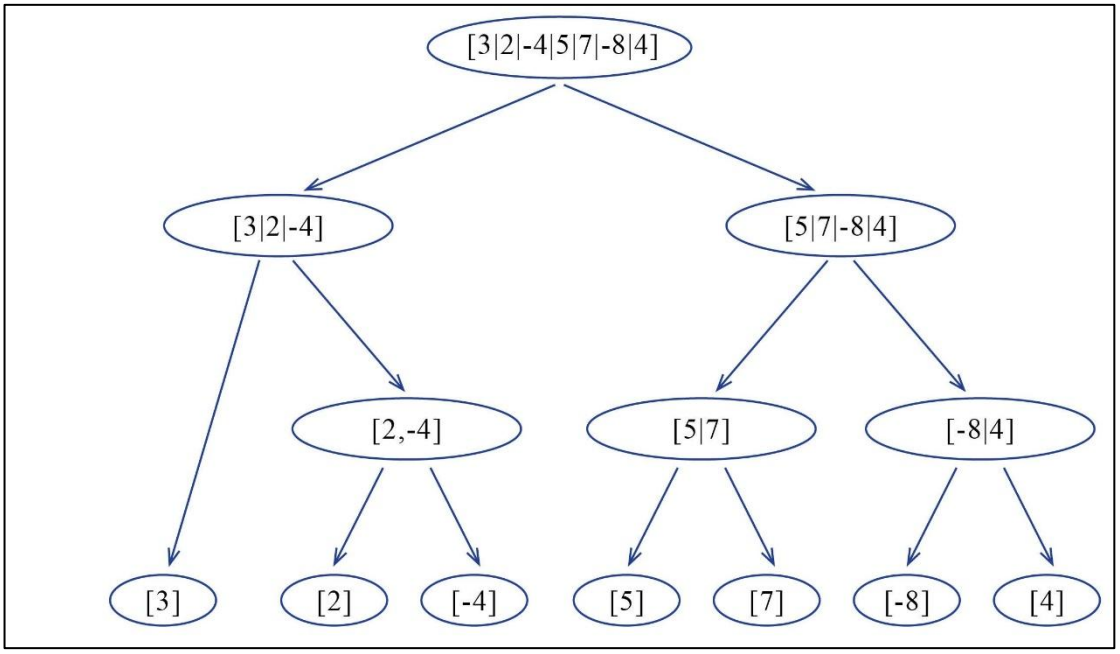
<u>דוגמאות</u>

נראה שלוש דוגמאות:

1. דוגמה לעץ הרקורסיה המושרה על ידי קלט נתון בן 7 איברים.
2. עץ רקורסיה כללי המייצג את כל הקלטים שאורכם 7.
3. עץ רקורסיה כללי המייצג את כל הקלטים שאורכם מספר שלם n.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה מס' 1: עץ הרקורסיה המושרה על ידי ביצוע אלגוריתם *Rsum* עם הקלט $[3,2,-4,5,7,-8,4]$.



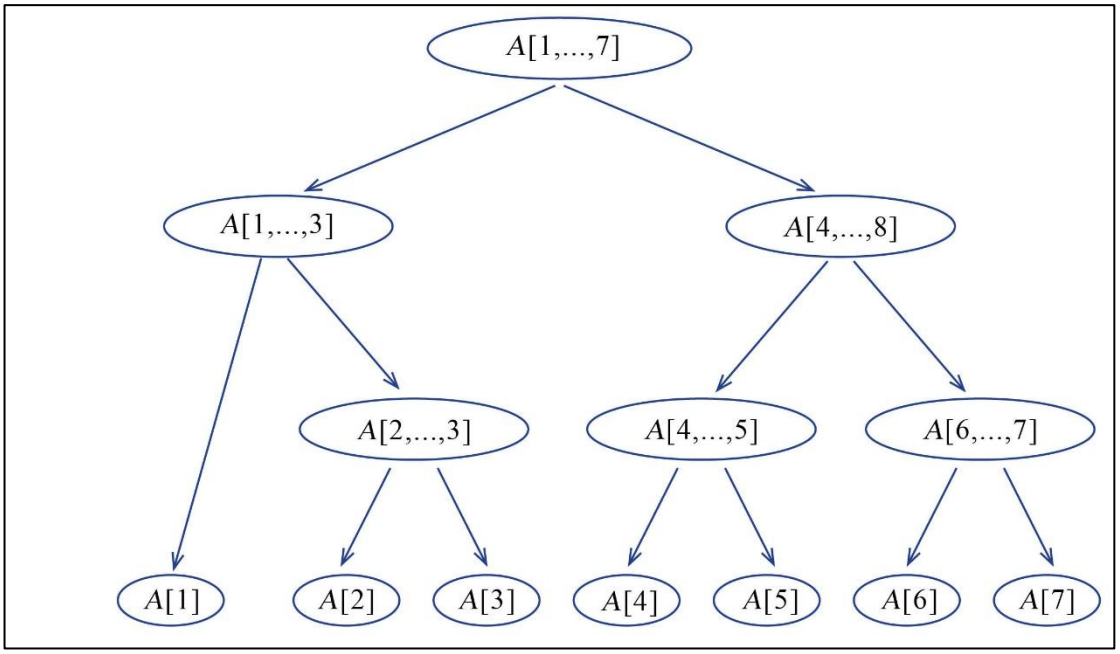
<u>איור 2.6: עץ הרקורסיה של *Rsum*</u>

<u>עם הקלט</u> $[3,2,-4,5,7,-8,4]$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה מס' 2: עץ רקורסיה של *Rsum* לקלט כללי בן 7 איברים

עץ הרקורסיה המוצג באיור 2.6 מייצג את ביצוע אלגוריתם *Rsum* עם מערך קלט כלשהו שבו 7 איברים $A[1,...,7]$:



איור 2.7: עץ רקורסיה כללי לקלט עם 8 איברים

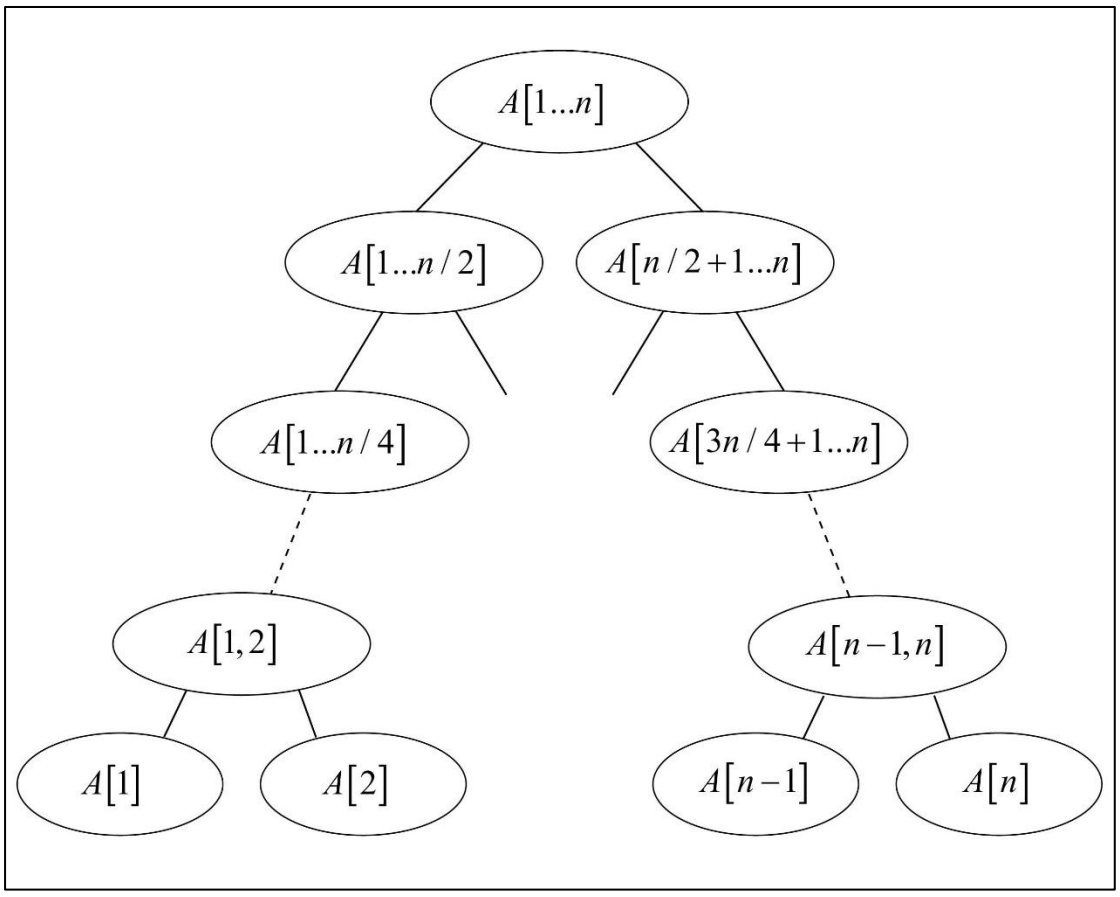
© כל הזכויות שמורות לפרופסור עמוס ישראלי

שימו לב: חלוקת הקלט לקלטי משנה אינה תלויה בערכים המאוחסנים במערך הקלט ולכן מבנהו של עץ הרקורסיה הכללי למערכים עם 7 איברים **דומה** למבנה עץ הרקורסיה עם הקלט $[3,2,-4,5,7,-8,4]$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

עץ רקורסיה של *Rsum* לקלט כללי

עץ הרקורסיה המוצג באיור 2.7 מייצג את ביצוע אלגוריתם *Rsum* עם מערך קלט כללי, $A[1,...,n]$.



איור 2.8: עץ הרקורסיה של אלגוריתם *Rsum* עם $A[1,\ldots,n]$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

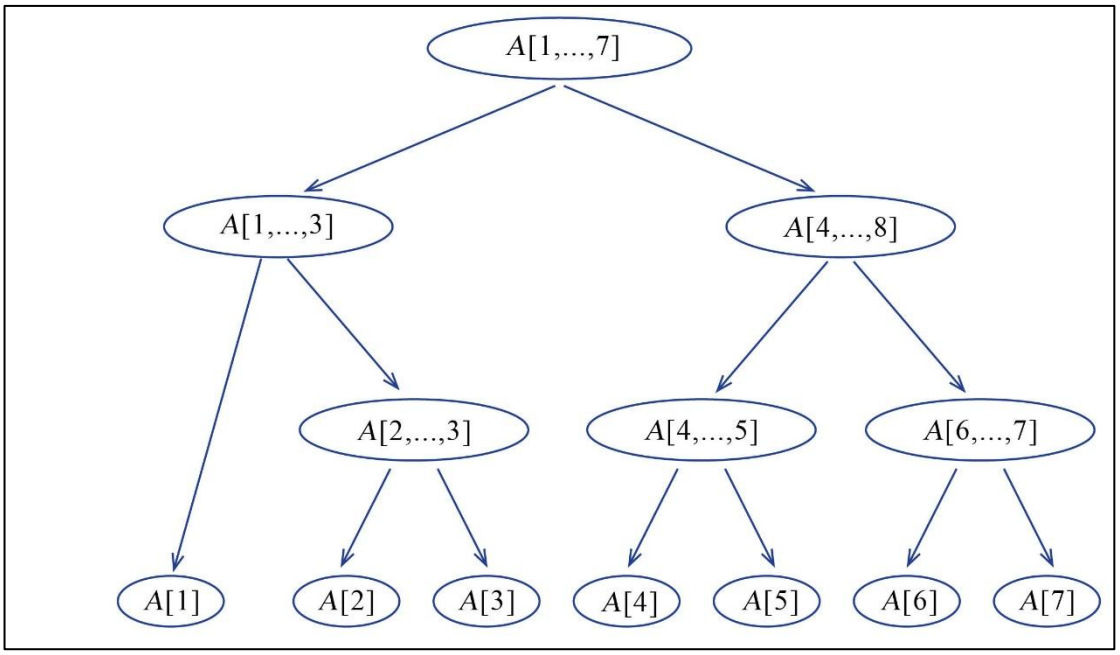
<u>שימו לב:</u>

נזכור שקבוצת הקלטים לאלגוריתם *Rsum* מכילה את כל מערכי המספרים.
עץ הרקורסיה מייצג את ביצוע האלגוריתם עם כל אחד ממערכי הקלט המותרים.
שורש העץ מייצג את כל המערכים בעלי n איברים. הבן השמאלי של השורש מייצג את המחצית השמאלית של מערך הקלט (עם אינדקסים $1,\ldots,\lfloor n/2 \rfloor$) והבן ימני מייצג את המחצית הימנית של מערך הקלט (עם אינדקסים $\lfloor n/2 \rfloor+1,\ldots,n$).
כל עלה בעץ מייצג תת-מערך עם איבר יחיד, אלו הם קלטי הקצה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שאלה

1. האם העץ הזה הוא **מלא?**
2. האם העץ הזה הוא **מאוזן?**



© כל הזכויות שמורות לפרופסור עמוס ישראלי

ניתח סיבוכיות אלגוריתם *Rsum*

באמצעות עץ הרקורסיה שלו

מספר צעדי החישוב המבוצעים על ידי כל קריאה עם קלט קצה (מערך עם איבר יחיד) הוא קבוע ואינו תלוי באיבר המאוחסן במערך הקלט. נסמן קבוע זה ב- c_1.

מאותה סיבה: מספר צעדי החישוב המבוצעים על ידי כל קריאה עם קלט שאינו קלט קצה (מערך עם יותר מאיבר יחיד) הוא קבוע ואינו תלוי באיברי מערך הקלט. נסמן קבוע זה ב- c_2.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

יהי T עץ רקורסיה כלשהו. נסמן ב-l את מספר העלים ב-T וב-k את מספר הצמתים הפנימיים ב-T ונקבל:

- מספר צעדי החישוב המבוצע על ידי כל קריאות הקצה הוא $l \cdot c_1$
- מספר צעדי החישוב המבוצע על ידי כל הקריאות שאינן קריאות קצה הוא $k \cdot c_2$

<u>מסקנה</u>

אפשר לחשב את סיבוכיות האלגוריתם תוך שימוש בעץ הרקורסיה שלו כמודגם בשקפים הבאים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיבוכיות *Rsum* - דוגמה לשימוש בעץ הרקורסיה

למה 2.9

יהי T עץ בינארי מלא ובו n עלים. ב-T יש $n-1$ צמתים פנימיים, כאשר כל צומת שיש לו בנים, נחשב כצומת פנימי.

הוכחה

ראו טענה 3 בנספח.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 2.10: סיבוכיות *Rsum*

סיבוכיות *Rsum* היא $\Theta(n)$.

הוכחה

נניח כי אלגוריתם *Rsum* נקרא עם מערך A באורך n. יהי T_n עץ הרקורסיה המתקבל. כל קריאה עם קלט שאינו מקרה קצה, מניבה 2 קריאות משנה ומכאן, T_n הוא עץ בינארי מלא (לכל צומת פנימי יש בדיוק 2 "בנים"). כל עלה של T_n מייצג קריאה ל-*Rsum* עם תת מערך של A ובו איבר יחיד וכל איבר כזה מופיע בדיוק בעלה אחד.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מכאן נובע כי ל-T_n יש בדיוק n עלים.

לפי למה 2.9, ל-T_n יש $n-1$ צמתים פנימיים.

נתבונן בקוד של $Rsum$: כל קריאה עם קלט קצה מבצעת מספר קבוע של צעדי חישוב שנסמנו ב-c_1, מאחר שיש n קלטי קצה, וכל אחד נקרא פעם אחת בלבד, הסיבוכיות הכוללת של מקרי הקצה היא $c_1 n$.

כל קריאה עם קלט אשר יש בו יותר מאיבר אחד, מבצעת מספר קבוע של צעדי חישוב. קבוע זה כולל את ביצוע **<u>הוראות הקריאה</u>** אך אינו כולל את ביצוע הפעולות של הקריאות

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הרקורסיביות כי סיבוכיות פעולות אלה מחושבת בצמתים המייצגים את הקריאות הרקורסיביות עצמן. בסך הכל מספר צעדי החישוב שקריאה כזו מבצעת הוא קבוע אחר שנסמנו ב-c_2.

מכאן, מספר צעדי החישוב הכולל ש-*Rsum* מבצע בחישוב עם A הוא

$$c_1 n + c_2 (n-1) = \Theta(n).$$ **מש"ל**

נעבור כעת ל"מתכון" כללי לחישוב סיבוכיות אלגוריתמים רקורסיביים. המתכון מבוסס על הדוגמה שהצגנו.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

"מתכון" לניתוח סיבוכיות אלגוריתמים רקורסיביים

נניח כי Alg הוא אלגוריתם רקורסיבי.

כדי לחשב את סיבוכיות Alg יש לבצע:

1. לחשב חסם על מספר הצעדים שכל קריאה ל-Alg מבצעת. אפשר לחשב לחוד את תרומת קריאות הקצה ואת תרומת הקריאות הרקורסיביות.
2. לחשב חסם על מספר הקריאות הכולל ל-Alg. בהערכה זו אנו מביאים בחשבון את הקריאה הראשית וכל אחת מן הקריאות הרקורסיביות.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בדרך כלל, עץ הרקורסיה מספק דרך נוחה לחישוב זה.

3. הכפלת החסם על מספר צעדי החישוב בכל קריאה בחסם על מספר הקריאות מניבה חסם על מספר צעדי Alg.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בחלקה השני של ההרצאה נציג את אלגוריתם **מיון באמצעות מיזוג (MergeSort).**

הגדרה אינטואיטיבית של בעיית המיון

אלגוריתם מיון מקבל כקלט מערך מספרים, ומחזיר מערך המכיל את אותם המספרים, מסודרים בסדר עולה או יורד.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרת תמורה

מערך B הוא **תמורה (Permutation)** של מערך A אם המערכים A ו-B מכילים **בדיוק אותם איברים** וכל איבר מופיע בדיוק אותו מספר פעמים ב-A וב-B.

איור 2.11: הגדרת תמורה

לדוגמה

1,2,3 היא תמורה של 3,1,2.

5,1,1 היא תמורה של 1,5,1

4,7,7 **אינה תמורה** של 4,4,7

3,2,7 **אינה תמורה** של 3,2,7,7

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה פורמלית של בעיית המיון

הקלט

מערך A בן n מספרים המופיעים בסדר שרירותי. למען הפשטות נניח כי כל איברי המערך A **נבדלים (distinct)**, כלומר: שונים זה מזה.

הפלט

מערך B ובו איברי A מסודרים לפי גודלם.

במלים אחרות: מערך הפלט הוא תמורה ממוינת של מערך הקלט.

איור 2.12: בעיית המיון

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מיון באמצעות מיזוג (Merge Sort)

נעבור כעת להצגת אלגוריתם **מיון באמצעות מיזוג (*MergeSort*)**.

אלגוריתם *MergeSort* משתמש באלגוריתם מיזוג כשיטת עזר.

קלט

שני מערכים ממוינים, A ו-B.

פלט

מערך C המכיל "תמורה" ממוינת של איברי A ו-B. (הכוונה למערך יחיד המכיל את כל איברי A ואת כל איברי B).

איור 2.13: בעיית המיזוג

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם מיזוג (Merge)

```
Merge(A,B)
    A[n+1] ← ∞ ; B[m+1] ← ∞
    ap ← 1; bp ← 1; cp ← 1;
    while (cp < m+n+1)
        if(A[ap] ≤ B[bp])
            C[cp] ← A[ap]; ap++
        else C[cp] ← B[bp]; bp++
        endif
        cp ← cp+1
    endwhile
    return(C)
end(Merge)
```

איור 2.14: אלגוריתם *Merge*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תיאור האלגוריתם

אלגוריתם *Merge* הוא אלגוריתם **איטרטיבי**, המקבל כקלט את המערכים A ו-B ומחזיר את המערך הממוזג C. בתחילת ביצוע האלגוריתם מאתחלים שלושה מצביעים (Pointers), *ap*, *bp* ו-*cp* כך שיצביעו אל האיבר הראשון במערכים A,Bו-C. בכל שלב של האלגוריתם משווים בין שני האיברים אליהם מכוונים *ap*, ו-*bp* ומעבירים את הקטן מבין השנים לאיבר עליו מכוון *cp*. לאחר מכן, מקדמים את המצביע שהצביע על

© כל הזכויות שמורות לפרופסור עמוס ישראלי

האיבר שהועבר ל-C, ואת המצביע של מערך C אל האיבר הבא.

כדי למנוע גלישה מעבר לגבולות A ו-B מוסיפים לכל אחד מהם את **איבר חוסם** שערכו ∞. הוספת האיברים החוסמים מפשטת את הצגת האלגוריתם. כאשר מממשים את האלגוריתם בתכנה, יש להשתמש במנגנון תכנתי מתאים כדי למנוע גלישה.

לדוגמה: לאחר כל ביצוע של הוראת ה-**if**, אפשר לבדוק האם המצביע המתאים חרג מן המערך המתאים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 2.15: נכונות אלגוריתם *Merge*

אלגוריתם *Merge* המקבל כקלט שני מערכים ממוינים בסדר עולה כלשהם, A ו-B, מחזיר מערך ממוין המכיל את איברי מערכי הקלט A ו-B.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחה

נתבונן בשמורה הבאה: לכל $1 \le i \le m+n$, לאחר i מעברים בלולאה הראשית, המערך $C[1,...,i]$ מכיל תמורה ממוינת של i האיברים הקטנים ביותר מבין איברי המערכים A ו-B, המצביעים ap, ו-bp, מצביעים על האיברים הראשונים ב-A , ו-B שטרם העברו ל-C, וערכו של cp הוא $i+1$. ההוכחה על ידי מעקב.

נכונות האלגוריתם נובעת מידית מנכונות השמורה לאחר $n+m$ מעברים בלולאה.

מש"ל

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיבוכיות אלגוריתם *merge*

משפט 2.16

סיבוכיות אלגוריתם המיזוג היא לינארית בסכום אורכי המערכים $t_{merge}(n,m) = \Theta(n+m)$.

הוכחה

מספר המעברים בלולאה הוא $\Theta(m+n)$ ומספר צעדי החישוב המתבצעים בכל מעבר הוא קבוע (Const).

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מיון מיזוג

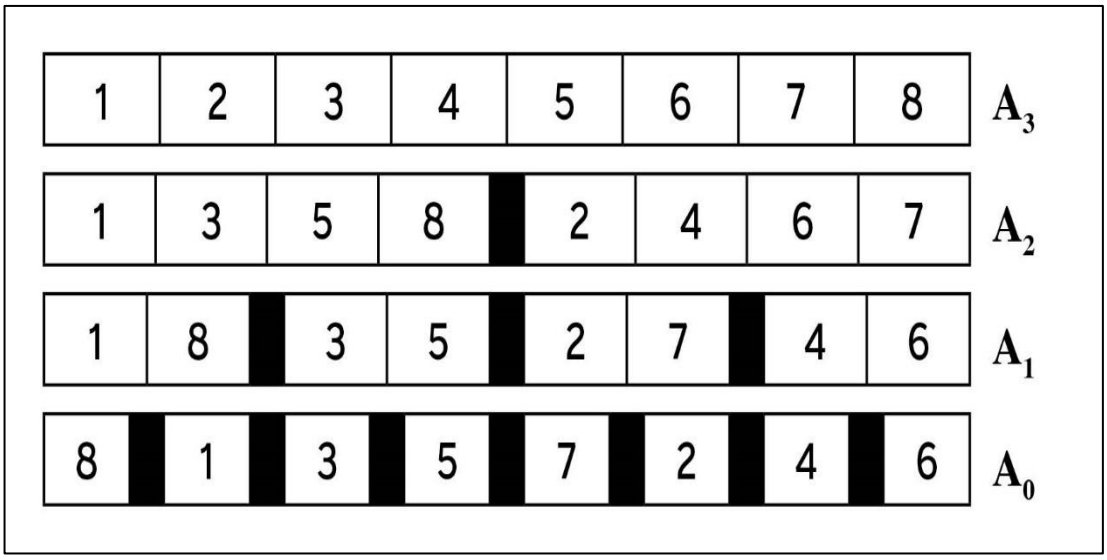
$$
\begin{array}{l}
MergeSort(A) \\
\quad \textbf{if}\ (|A|=1)\ return(A) \\
\quad mid \leftarrow \lfloor n/2 \rfloor \\
\quad B_l \Leftarrow MergeSort(A[1,\ldots,mid]) \\
\quad B_r \Leftarrow MergeSort(A[mid+1,\ldots,n]) \\
\quad C \Leftarrow merge(B_l, B_r) \\
\quad \textbf{return}(C) \\
\textbf{end}(MergeSort)
\end{array}
$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

נעבור כעת לתיאור אלגוריתם מיון מיזוג. לצורך פשטות התיאור, נניח שמספר איברי מערך הקלט הוא חזקה שלמה של 2, כלומר $|A| = n = 2^m$.

במהלך האלגוריתם מחושבים $m+1$ מערכי ביניים, $A_0, \ldots, A_m$, אשר כל אחד מהם הוא תמורה של מערך הקלט. באיור 2.17 מוצגים ארבעת מערכי הביניים הנוצרים במהלך מיון מיזוג של מערך בן 8 איברים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי



איור 2.17: סדרת מערכי הבינים למיון מיזוג של מערך בן 8 איברים

1. המערך A_0 הוא מערך הקלט. אנו נסתכל על A_0 כסדרה של n תת-מערכים ממוינים (כי בכל אחד מהם יש איבר יחיד).

© כל הזכויות שמורות לפרופסור עמוס ישראלי

2. המערך A_1 מתקבל מ-A_0 על ידי מיזוג של כל שני מערכים שכנים. אפשר לראות שתחת ההנחה ש-$n = 2^m$, המערך A_1 מכיל $n/2$ תת מערכים ממוינים שבכל אחד מהם $2^1 = 2$ איברים.
3. לכל $2 \le i \le m$, A_i מתקבל מ-A_{i-1} על ידי מיזוג של כל שני תת מערכים שכנים. מספר תת המערכים ב-A_i הוא $n/2^i$ ובכל תת מערך יש 2^i איברים.
4. לאחר ביצוע $m = \log n$ מעברים כאלה נקבל ש-A_m ממוין. בסיום הביצוע, A_m מוחזר כפלט.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם איטרטיבי או רקורסיבי

חישוב המערכים $A_0,\ldots,A_m$ ניתן לביצוע הן באמצעות אלגוריתם איטרטיבי והן באמצעות אלגוריתם רקורסיבי.
הקוראת האינטליגנטית מתבקשת לעצור לרגע ולנסות לבחור באופן המימוש.

התשובה

כאשר מספר איברי מערך הקלט שונה מחזקה שלמה של 2, המימוש האיטרטיבי דורש חישובים, מסובכים למדי, של גבולות הלולאות באיטרציות השונות.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בנוסף הוכחת הנכונות של אלגוריתם איטרטיבי צפויה להיות מסובכת למדי. האלגוריתם הרקורסיבי מבצע את החלוקה לתת מערכים באמצעות מנגנון הרקורסיה ובכך חוסך טרחה רבה מן המתכנת. הוכחת הנכונות גם היא אלגנטית בהשוואה להוכחת הנכונות של האלגוריתם האיטרטיבי. מסיבות אלה המימוש הרקורסיבי הוא המימוש המקובל למיון מיזוג.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

המימוש הרקורסיבי

```
MergeSort(A)
    if (|A|=1) return(A)
    mid ← ⌊n/2⌋
    B_l ⇐ MergeSort(A[1,...,mid])
    B_r ⇐ MergeSort(A[mid+1,...,n])
    C ⇐ merge(B_l, B_r)
    return(C)
end(MergeSort)
```

איור 2.16: מיון מיזוג (*MergeSort*)

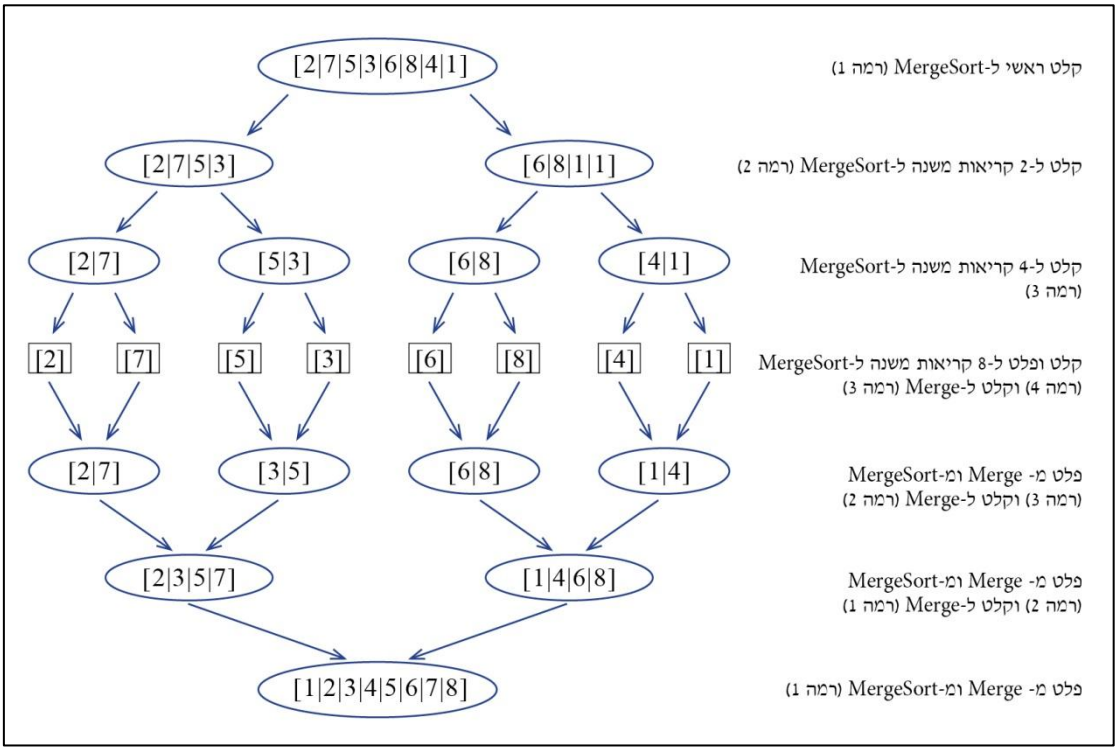
© כל הזכויות שמורות לפרופסור עמוס ישראלי

תיאור מיון מיזוג כאלגוריתם רקורסיבי

- בתחילת הביצוע מיון מיזוג בודק האם מערך הקלט מכיל איבר יחיד, כלומר האם הקלט הוא קלט קצה. אם התשובה חיובית, הקלט מחזר כפלט.
- אם הקלט אינו קלט קצה מיון מיזוג מבצע שתי קריאות משנה. האחת עם תת המערך השמאלי $A_l = A[1,...,\lfloor n/2 \rfloor]$ והשנייה עם תת המערך הימני.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

- מערך הפלט מתקבל על ידי מיזוג מערכי הפלט המתקבלים מן הקריאות הרקורסיביות באמצעות אלגוריתם *Merge*.



איור 2.17: תיאור קלט ופלט של *MergeSort*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

נכונות מיון מיזוג

עלינו להוכיח:

1. מיון מיזוג עוצר (ביצועו מסתיים).
2. מיון מיזוג מחזיר פלט נכון.

משפט 2.18: סיום מיון מיזוג

נניח שמיון מיזוג מבוצע עם מערך מספרים A. בכל קריאה רקורסיבית הקלט קטן פי 2. בסיום כל שרשרת קריאות, גודל הקלט מגיע ל-1 וזהו קלט קצה אשר אינו מבצע קריאות רקורסיביות נוספות. מכאן נובע שכל שרשרת קריאות רקורסיביות מסתיימת ולכן גם האלגוריתם כולו מסתיים. **מש"ל**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 2.19: נכונות מיון מיזוג

לכל $n > 0$, אלגוריתם מיון מיזוג ממיין כל מערך מספרים באורך n .

הוכחה

נוכיח את המשפט באינדוקציה שלמה על **אורך הקלט** n.

בסיס ($n = 1$)

הקלט הוא מערך ובו איבר יחיד. כל מערך בן איבר יחיד הוא ממוין באופן טריוויאלי.

.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הנחת האינדוקציה

נניח כי מיון מיזוג הנקרא עם מערך קלט A שמספר איבריו גדול מ-1 וקטן (ממש) מ-n מחזיר תמורה ממוינת של A (משתמשים באינדוקציה שלמה).

צעד האינדוקציה

נניח כי אלגוריתם $MergeSort$ נקרא עם קלט שארכו n, $n > 1$.

בתחילת האלגוריתם, מערך הקלט מחולק ל**שני** קלטי משנה: A_l, שארכו $\lfloor n/2 \rfloor$ ו-A_r שארכו $n - \lfloor n/2 \rfloor$. ארכו של כל קלט משנה **קטן ממש** מ-n ולכן, לפי הנחת האינדוקציה, לאחר ביצוע

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הקריאות הרקורסיביות כל אחד ממערכי הפלט, B_l ו-B_r, מאחסן תמורה ממוינת של מערך הקלט שלו וקבוצת איבריהם של מערכי הפלט שווה לקבוצת איברי מערך הקלט.

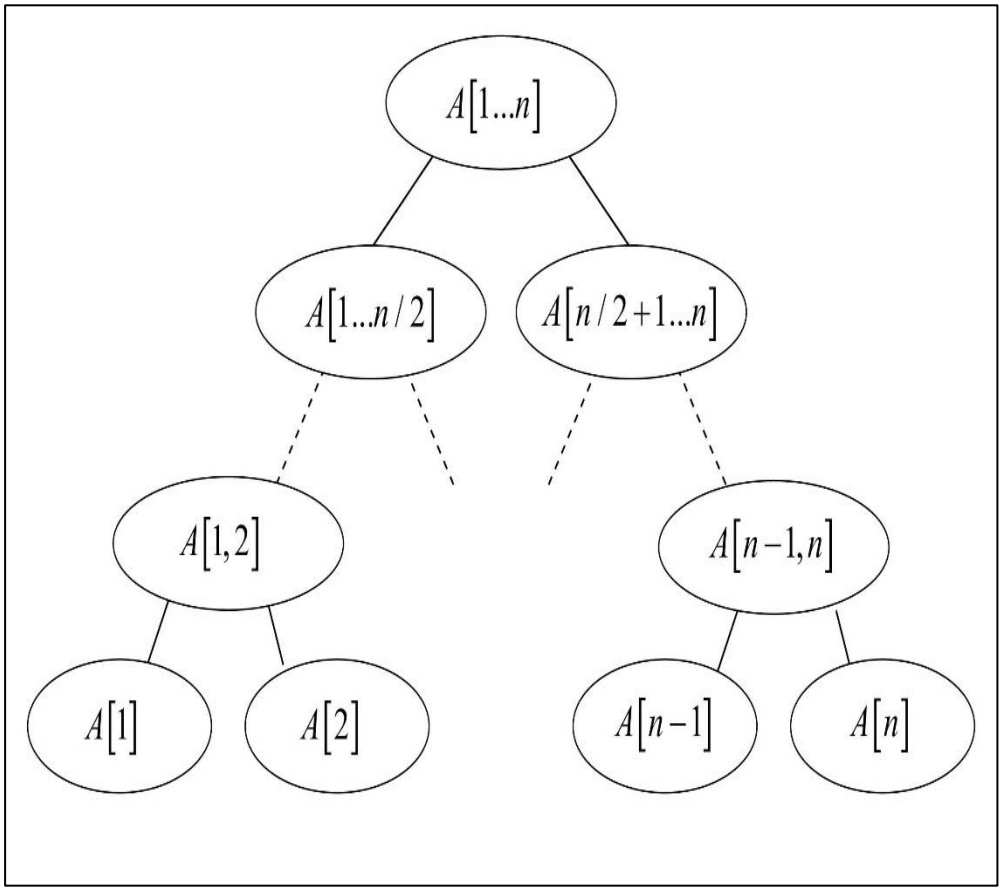
בהמשך, השיטה *merge* נקראת עם המערכים B_l ו-B_r. **שימו לב:**

המערכים B_l ו-B_r כבר ממוינים, ולכן השיטה *Merge* מחזירה תמורה ממוינת של מערך הקלט. **מש"ל**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

עץ הרקורסיה של מיון מיזוג

עץ הרקורסיה מושרה על ידי מיון מיזוג מוצג באיור 2.20



איור 2.20: עץ הרקורסיה של מיון מיזוג

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיבוכיות מיון מיזוג

נניח כי *MergeSort* מתבצע עם מערך A ובו n איברים. כאשר $n = 2^m$ הוא חזקה שלמה של 2.

נסמן ב- T_n את עץ הרקורסיה: בכל ביצוע של האלגוריתם מתבצעות שתי קריאות רקורסיביות; מכאן, T_n הוא עץ בינארי מלא. לפי טענה 3 בנספח ב- T_n יש $2n-1$ צמתים.

נתבונן בקוד של מיון מיזוג: כל אחת מן ההוראות בקוד (כולל הקריאות הרקורסיביות) **למעט הוראות ה-*Merge*** מתבצעת בזמן קבוע. מכאן נובע שזמן ביצוע כל הוראות אלגוריתם

© כל הזכויות שמורות לפרופסור עמוס ישראלי

MergeSort עם A, **למעט זמני הביצוע של השיטה *merge***, הוא לינארי.

נעבור להערכת מספר צעדי החישוב המבוצעים על ידי כל הקריאות לשיטה *merge*.

ב-T_n יש $m+1$ רמות הממוספרות מ-0 (רמת השורש) ועד m (רמת העלים).

ברמה m לא מתבצעים מיזוגים כלל.

ברמה $m-1$ (ההורים של העלים) יש $n/2$ צמתים. בכל צומת ברמה זו מתבצעת הוראת *merge* אחת הממזגת שני מערכים שבכל אחד מהם יש איבר יחיד. נזכור שסיבוכיות אלגוריתם

© כל הזכויות שמורות לפרופסור עמוס ישראלי

merge שווה למספר האיברים במערך הממוזג. מכאן נובע שהסיבוכיות הכוללת של הוראות ה-*merge* המתבצעות ברמה $m-1$ היא $\Theta(n)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

כעת נתבונן ברמה i, $0 < i \le m-1$, של T_n: ברמה זו יש 2^i צמתים. אפשר גם לראות שבכל צומת מתבצעת הוראת $Merge$ אחת ובה ממזגים שני מערכים שבכל אחד מהם יש $n/2^{i+1}$. מכאן נקבל שמספר ההוראות הכולל המתבצעות בקריאות ל-$Merge$ ברמה i הוא

$$2 \cdot 2^i \cdot n/2^{i+1} = n.$$

בעץ T_n יש בדיוק $m = \log n$ רמות ומכאן נובע שמספר ההוראות הכולל המתבצע על ידי השיטה $Merge$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בביצוע עם A הוא $\Theta\left(n\log n\right)$ וזוהי סיבוכיות מיון מיזוג עבור כל מערך שמספר איבריו הוא חזקה שלמה של 2.

כעת נרחיב טענה זו לכל n.

נניח כעת שמיון מיזוג מופעל עם מערך קלט A אשר מספר איבריו הוא n ו-n אינו חזקה שלמה של 2. נניח שמתקיים $2^m < n < 2^{m+1}$ (תמיד יש m כזה).

לפי מה שהוכחנו קודם סיבוכיות מיון מיזוג עם כל מערך שבו יש 2^m איברים היא $\Theta\left(2^m m\right)$ וסיבוכיות מיון מיזוג

© כל הזכויות שמורות לפרופסור עמוס ישראלי

עם מערך שיש בו 2^{m+1} איברים היא

$$\Theta\left(\left(2^{m+1}\right)m+1\right).$$

נזכור שמתקיים $2^m < n < 2^{m+1}$, נסמן ב-η את סיבוכיות מיון מיזוג עם מערך בן n איברים ונסיק:

$$\Theta\left(m2^m\right) < \eta < \Theta\left(\left(m+1\right)2^{m+1}\right)$$

כעת נפתח את הביטוי $\left(m+1\right)2^{m+1}$ ונקבל

$$m2^{m+1} + 2^{m+1} = 2m^m + 2n =$$

$$= 2n\log n + 2n = \Theta\left(n\log n\right)$$

מכאן נובע

© כל הזכויות שמורות לפרופסור עמוס ישראלי

$$\Theta\left(n\log n\right)\le n\le\Theta\left(n\log n\right)$$

כלומר לכל n טבעי סיבוכיות מיון מיזוג עם מערך ובו n איברים היא $\Theta\left(n\log n\right)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיכום ההרצאה

פתחנו את ההרצאה בהדגמת פיתוח אלגוריתמים רקורסיבי *Rsum*, לפתרון בעיית סכום ההמשכנו את ההדגמה בהוכחת נכונות *Rsum* ובניתוח הסיבוכיות שלו.

לאחר מכן, הצגנו "מתכונים" לביצוע פעולות אלה באופן כללי.

בחלקה השני של ההרצאה, הצגנו את אלגוריתם **מיון מיזוג**, הוכחנו את נכונותו וניתחנו את הסיבוכיות שלו בעזרת השיטות שהצגנו.

סיבוכיות אלגוריתם מיון מיזוג היא $\Theta(n\log n)$. זהו אלגוריתם המיון

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שהסיבוכיות שלו היא הטובה ביותר מבין כל האלגוריתמים שנכיר. בהמשך הקורס נוכיח כי זוהי הסיבוכיות האופטימלית עבור כל אלגוריתם מיון **מבוסס השוואות**.

אלגוריתם מיון נוסף המשיג סיבוכיות זו הוא **מיון ערמה (Heap Sort)** שאותו לא נלמד בקורס.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תרגיל

נתון המערך

[7,5,12,43,9,-3,8,-16,22] .

הציגו את מערכי המשנה המחושבים במהלך ביצוע האלגוריתם.

הציגו את עץ הרקורסיה המייצג את ביצוע האלגוריתם.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

נספח: עצים בינאריים

נפתח בחזרה על כמה מושגים הקשורים לעצים בינאריים כלליים:

1. **צומת** - רשומה המכילה נתונים ומצביעים אל צמתים אחרים. כל מצביע לצומת אחרת נקרא **קשת.** צומת שלא יוצאת ממנו קשת נקרא **עלה**.
2. **עץ בינארי** - מבנה נתונים המכיל צמתים וקשתות. **שורש העץ** r הוא צומת אליו לא נכנסת אף קשת. לכל צומת בעץ הבינארי, למעט השורש יש **אב** יחיד. לכל צומת יש 0-2 **בנים.** אחד הבנים של הצומת יקרא בן **שמאלי**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

והשני בן **ימני**. אם לצומת יש רק בן אחד, הוא יכול להיות שמאלי או ימני.

3. הבן השמאלי של שורש העץ T (אם הוא קיים) הוא **השורש** של **תת העץ השמאלי** של T.
4. הבן הימני של שורש העץ T (אם הוא קיים) הוא **השורש** של **תת העץ הימני** של T.
5. **רמה** של צומת היא **מספר הקשתות** במסלול (היחיד) מן השורש אל הצומת.
6. **עלה** - צומת שאין לו בנים.
7. **גובה** של עץ - רמה מכסימלית של עלה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

8. **עץ בינארי מלא** - עץ בינארי בו לכל צומת שאינו עלה יש **שני בנים.**
9. **עץ בינארי מאוזן** - עץ בינארי מלא בו כל העלים נמצאים באותה רמה.

עץ בינארי כמעט מאוזן - עץ בינארי בו כל הרמות מלאות למעט הרמה האחרונה. הרמה האחרונה מלאה משמאל עד לנקודה כלשהי.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תכונות עצים בינאריים

טענה 1

יהי T_1 עץ בינארי מאוזן בגובה h. אזי מתקיים:

1. מספר הצמתים ברמה l, $l = 0,1,...,h$ הוא 2^l.
2. מספר הצמתים ב T_1 הוא $2^{h+1} - 1$.

הוכחת טענה 1

תחילה נוכיח את 1 באינדוקציה על l.

בסיס: $l = 0$ - ברמה 0 יש צומת יחיד ואכן מתקיים .

© כל הזכויות שמורות לפרופסור עמוס ישראלי

צעד: נניח כי ברמה l יש 2^l צמתים.

העץ T_1 מאוזן, ולכן, לכל צומת ברמה l יש בדיוק שני בנים, ברמה $l+1$. מכאן נובע כי מספר הצמתים ברמה $l+1$ של T_1 הוא $2 \cdot 2^l = 2^{l+1}$.

כדי להוכיח את סעיף 2 נשים לב כי אם n הוא מספר צמתי T_1 אזי מתקיים:

$$n = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

טענה 2

יהי T_2 עץ בינארי כמעט מאוזן שגבהו h. נסמן את מספר איברי T_2 ב - n. אזי n מקיים:

$$2^h \le n \le 2^{h+1} - 1$$

מכאן נובע כי:

$$h \le \log n < h + 1$$

או

$$h = \lfloor \log n \rfloor$$

הוכחה

מיידית.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

טענה 3 (למה 2.7)

יהי T עץ בינארי מלא ובו n עלים. ב-T יש $n-1$ צמתים פנימיים, כאשר כל צומת שיש לו בנים, נחשב כצומת פנימי.

הוכחה

נוכיח את הלמה באינדוקציה על n.

בסיס – ($n=1$)

העץ הבינארי היחיד המקיים את התנאי הוא עץ בו השורש הוא הצומת (והעלה) היחיד. במקרה זה, מספר הצמתים הפנימיים הוא 0. **מש"ל**

(והעלה) היחיד. במקרה זה, מספר הצמתים הפנימיים הוא 0. **מש"ל**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

צעד האינדוקציה

נסמן את מספר הצמתים הפנימיים ב-T ב$ni(T)$. נניח כי הלמה נכונה לכל עץ בינארי מלא שמספר איבריו קטן או שווה ל-n (נשתמש באינדוקציה מלאה). יהי T עץ בינארי מלא ובו $n+1$ עלים, ויהיו T_l, ו-T_r תת העצים השמאלי והימיני של T. נסמן ב-n_l את מספר העלים ב-T_l, $1 \le n_l < n$. ב-T_r יש $n - n_r$ עלים. לפי הנחת האינדוקציה,

$$ni(T_l) = n_l - 1,$$

ו-$$ni(T_r) = (n - n_l) - 1.$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

כדי לחשב את $ni(T)$ יש לסכום את מספר הצמתים הפנימיים ב-T_l, עם מספר הצמתים הפנימיים ב-T_r, ולהוסיף 1, עבור השורש של T, כלומר:

$$ni(T) = ni(T_l) + ni(T_r) + 1 = n - 1 + 1 = n$$

מש"ל

© כל הזכויות שמורות לפרופסור עמוס ישראלי